캡스톤디자인 1조 · 물어봐 캠퍼스

물어봐 캠퍼스 API 명세서 v1.0

- 작성일: 2026-10-12 · 캡스톤디자인 1조 (송승현·이승우)
- 기준 문서: 요구사항 정의서 v1.11, 화면설계서 v1.5, DB 모델링 v1.0 (`05.1조_Ask Campus_DB모델링.pptx`)
- 요구사항 FR-019는 "API 요청·응답 형식은 JSON으로 하며, 상세 명세는 설계 단계에서 정의한다"고 정했습니다. 이 문서가 그 명세입니다.
- 표의 테이블 이름은 DB 모델링의 테이블과 같습니다.

0. 공통 규칙

항목	규칙
주소	앱 API `/api/v1/…` , 관리 웹 API `/admin/api/…` . 관리 웹 화면(Jinja2)은 `/admin/…` 이며 화면이 아래 관리 웹 API 를 호출
형식	요청·응답 모두 JSON (UTF-8). 보충 문서 업로드만 `multipart/form-data` , CSV 내려받기만 `text/csv`
시간	한국 표준시(KST) ISO 8601 문자열 (예: `2026-10-12T09:00:00+09:00`)
학생 인증	없음 (로그인 없이 익명 이용, 2.6). 이름·학번·기기 ID를 받지 않음
관리 웹 인증	이메일 인증 코드로 로그인하면 세션 쿠키 발급 (`HttpOnly` , `SameSite=Lax` , 12시간). 쿠키가 없거나 만료되면 401
권한	🔵 중간 관리자(학과 조교)·관리자 모두 사용 / 🔴 관리자만 사용 (중간 관리자가 부르면 403)
요청 제한	질문은 1~200자. 앱 API는 같은 접속 기준 분당 20회 (메모리에서만 세고 저장하지 않음)
응답 시간	질문 답변 평균 5초 이내 (PER-001). 앱은 10초 동안 응답이 없으면 재시도 버튼 표시 (AVL-003)

공통 오류 형식

```
{ "error": { "code": "INVALID_QUESTION", "message": "질문은 1~200자로 입력해 주세요." } }
```

HTTP	code	언제
400	`INVALID_QUESTION`	질문이 비었거나 200자 초과
400	`INVALID_INPUT`	그 밖의 입력 형식 오류 (이메일 형식, 파일 형식·크기 등)
401	`UNAUTHORIZED`	관리 웹 세션이 없거나 만료됨
401	`INVALID_CODE`	인증 코드가 틀리거나 만료·사용됨
403	`FORBIDDEN`	중간 관리자가 관리자 전용 기능 호출, 또는 고정 관리자 삭제 시도
404	`NOT_FOUND`	대상이 없음
409	`CONFLICT`	이미 등록된 이메일·동의어, 이미 답변한 질문
423	`LOCKED`	인증 코드 5회 실패로 10분 동안 로그인 제한
429	`RATE_LIMITED`	요청 제한 초과 (질문 분당 20회, 인증 코드 1분 1회)
503	`MODEL_LOADING`	서버가 켜진 직후 모델을 불러오는 중

1. 앱 API (학생 앱)

#	메서드·경로	용도	화면	요구사항
1-1	`GET /api/v1/health`	서버 상태 확인	—	FR-019
1-2	`POST /api/v1/ask`	질문하고 답변 받기	SCR-001, 001-A, 001-B	FR-001~005, 007, 022, 023, 027, 028
1-3	`GET /api/v1/deadlines`	마감 임박 일정 목록	SCR-005	FR-020
1-4	`GET /api/v1/faqs/recommended`	카테고리별 추천 질문	SCR-001	FR-006
1-5	`GET /api/v1/faqs/{faq_id}`	답변 알림을 눌렀을 때 질문과 답변	SCR-001-C	FR-030

1-1. `GET /api/v1/health`

```
{ "status": "ok", "model_ready": true, "data_updated_at": "2026-10-12T09:00:12+09:00" }
```

`data_updated_at`은 설정 화면(SCR-004)의 '데이터 기준일'에 씁니다.

1-2. `POST /api/v1/ask`

요청

필드	타입	필수	설명
`question`	string	O	질문 (1~200자)
`category`	string		고른 카테고리 칩: `학사정보` / `통학버스` / `동아리·행사` / `교외활동`
`campus`	string	O	설정 캠퍼스: `영동` / `아산`
`interests`	string[]		설정한 학과·관심 분야 (서버에 저장하지 않음)
`fcm_token`	string		알림 권한을 허용한 경우의 익명 기기 토큰. 근거가 없을 때만 `notify_token`에 저장
`clarify`	object		되묻기에 답할 때만: `{ "original_question": "...", "choice": "AI·SW" }`

```
{
  "question": "지원할 만한 공모전 있어?",
  "category": "교외활동",
  "campus": "아산",
  "interests": [],
  "fcm_token": "dK3…(익명 토큰)"
}
```

응답 200. `answer_type`에 따라 채워지는 필드가 다릅니다.

필드	타입	설명
`answer_type`	string	`문서` / `FAQ` / `되묻기` / `근거없음`
`answer`	string	답변 문장 (존댓말). 근거없음이면 안내 문구
`sources`	object[]	출처 최대 3개 (유사도 순, FR-004): `title`, `kind` (`공지` / `학과 조교 답변` / `보충 문서`), `source_url`, `menu_path`, `collected_at`
`schedule`	object|null	일정 답변이면 '관심 일정 저장'용: `chunk_id`, `title`, `deadline`, `source_url`

필드	타입	설명
`clarify`	object|null	되묻기면 `question`과 `options` (항상 `전체` 포함, FR-027)
`pending_registered`	boolean	근거가 없어 답변 대기에 자동 등록됐는지 (FR-028)
`elapsed_ms`	integer	서버 처리 시간

되묻기 예

```
{
  "answer_type": "되묻기",
  "answer": "어떤 분야를 찾으세요? 학과나 관심 분야를 알려주시면 맞춰 드릴게요.",
  "sources": [], "schedule": null,
  "clarify": { "question": "어떤 분야를 찾으세요?", "options": ["AI·SW", "디자인", "경영·마케팅", "전체"] },
  "pending_registered": false, "elapsed_ms": 820
}
```

근거없음 예

```
{
  "answer_type": "근거없음",
  "answer": "해당 정보를 찾을 수 없습니다. 아직 답을 모르는 질문이라 학과 조교에게 전달했어요. 답변이 등록되면 알림으로
알려 드릴게요.",
  "sources": [], "schedule": null, "clarify": null,
  "pending_registered": true, "elapsed_ms": 2140
}
```

서버 처리 순서

1. 질문 길이·요청 제한 확인 → 동의어 사전(`synonym`)으로 줄임말을 바꿈 (예: 수정 → 수강정정)
2. 조건이 빠졌으면 되묻기로 응답 (한 질문에 최대 1회)
3. 질문 임베딩 → 벡터 DB에서 FAQ와 청크를 상위 5개 검색 (카테고리·캠퍼스 필터)
4. 유사도가 기준 이상이면 sLLM이 근거와 현재 날짜·시각(KST)으로 답변 생성
5. 기준 미만이면 `pending_question`에 자동 등록 (비슷한 질문이면 `question_count` + 1). `fcm_token`이 있으면 `notify_token`에 추가 (같은 기기의 같은 질문은 1회만)
6. `query_log`에 질문 문장(이름·학번·전화번호 형태는 가림)·답변 유형·처리 시간만 기록

1-3. `GET /api/v1/deadlines`

쿼리	설명
`campus`	`영동` / `아산` (필수)
`type`	`전체` (기본) / `학사` / `교외활동`. '관심' 필터는 기기에 저장한 일정으로 앱이 거름
`days`	오늘부터 며칠 안의 마감까지 (기본 60)

```
{
  "items": [
    { "chunk_id": "notice-2611-0", "title": "2학기 수강정정 마감", "category": "학사정보",
      "deadline": "2026-09-30T18:00:00+09:00", "d_day": 1, "source_url": "https://…", "posted_at": "2026-09-24T1
0:00:00+09:00" }
  ]
}
```

`chunk.deadline`이 있는 항목만 마감 순으로 보냅니다 (추출이 불확실한 항목은 NULL이라 빠짐, FR-020).

1-4. `GET /api/v1/faqs/recommended`

쿼리: `category` (선택), `campus` (필수), `limit` (기본 4). `faq.ask_count` 가 많은 순입니다.

```
{ "items": [ { "faq_id": 31, "question": "수강정정 언제까지야?" }, { "faq_id": 12, "question": "이번 주 학사일정 알려줘" } ] }
```

1-5. `GET /api/v1/faqs/{faq_id}`

```
{
  "faq_id": 57, "question": "편입생 전공 학점 인정 몇 학점까지?",
  "answer": "편입생의 전공 학점 인정은 학과 안내 기준에 따라 최대 OO학점까지 인정됩니다. …",
  "category": "학사정보", "source_label": "학과 조교 답변 · 2026-10-09",
  "ref_doc": { "title": "편입생 학점 인정 안내", "url": null }
}
```

답변 알림 (FCM 데이터 메시지, 서버 → 앱)

```
{ "type": "answer_registered", "faq_id": "57", "title": "물어봐 캠퍼스", "body": "물어본 질문에 답변이 등록되었어요" }
```

앱은 알림을 누르면 1-5를 호출해 SCR-001-C를 보여 줍니다. 서버는 보낸 뒤 해당 `notify_token` 행을 지웁니다 (FR-030).

2. 관리 웹 API (학과 조교·관리자)

#	메서드·경로	권한	용도	화면	요구사항
2-1	`POST /admin/api/auth/code`	누구나	인증 코드 받기	SCR-006-A	FR-033
2-2	`POST /admin/api/auth/login`	누구나	코드로 로그인	SCR-006-A	FR-033
2-3	`POST /admin/api/auth/logout`	🔵	로그아웃	공통	FR-033
2-4	`GET /admin/api/dashboard`	🔵	운영 지표	SCR-006	FR-025
2-5	`GET /admin/api/pending`	🔵	답변 대기 목록	SCR-006	FR-025, 028
2-6	`POST /admin/api/pending/{id}/answer`	🔵	답변 작성·즉시 반영·알림	SCR-006-B	FR-029, 030
2-7	`POST /admin/api/pending/{id}/not-mine`	🔵	담당 아님 표시	SCR-006, 006-B	FR-029
2-8	`GET·PUT·DELETE /admin/api/faqs…`	🔵	FAQ 조회·수정·삭제	SCR-006 (FAQ 관리)	FR-024, 029
2-9	`GET·POST·DELETE /admin/api/managers…`	🔴	중간 관리자 이메일 관리	SCR-006-D	FR-032
2-10	`GET·POST·DELETE /admin/api/documents…`	🔴	보충 문서 업로드	SCR-006-C	FR-016, 024
2-11	`GET·POST·DELETE /admin/api/synonyms…`	🔴	동의어 사전	SCR-006	FR-002, 024
2-12	`GET /admin/api/collect/status`, `POST /admin/api/collect/run`	🔴	수집 현황·수동 실행	SCR-006 (수집 현황)	FR-014, 024
2-13	`GET /admin/api/pending.csv`, `GET /admin/api/faqs.csv`	🔵	CSV 내려받기	SCR-006	SCR-006 인쇄 조건

2-1. `POST /admin/api/auth/code`

요청 `{ "email": "assist@u1.ac.kr" }` → 응답 200 (등록 여부와 관계없이 같은 문구, FR-033)

```
{ "message": "등록된 이메일이면 인증 코드를 보냈습니다." }
```

- 등록된 이메일이면 6자리 코드를 만들어 해시만 `auth_code`에 저장하고 메일로 보냄 (1분 이내 발송, PER-011)
- 같은 이메일로 1분 안에 다시 요청하면 429

2-2. `POST /admin/api/auth/login`

요청 `{ "email": "assist@u1.ac.kr", "code": "381042" }` → 응답 200 + 세션 쿠키

```
{ "role": "중간 관리자", "name": "김조교", "expires_at": "2026-10-12T21:30:00+09:00" }
```

- 틀리면 401 `INVALID_CODE` (`fail_count` + 1), 5회 틀리면 423 `LOCKED` (10분)
- 성공하면 코드에 `used_at`을 기록하고 `login_session` 생성, `manager.last_login_at` 갱신

2-4. `GET /admin/api/dashboard`

```
{ "today_questions": 128, "pending_count": 6, "avg_response_ms": 3400,
  "daily": [ { "date": "2026-10-11", "questions": 97, "no_answer": 4 } ] }
```

`query_log`와 `pending_question`에서 집계합니다 (최근 7일).

2-5. `GET /admin/api/pending`

쿼리: `status`(기본 `대기`), `category`. 같은 질문 수가 많은 순입니다.

```
{ "items": [ { "pending_id": 14, "question": "편입생 전공 학점 인정 몇 학점까지?", "category": "학사정보",
               "campus": "아산", "question_count": 14, "notify_count": 14,
               "first_asked_at": "2026-10-07T13:20:00+09:00", "expires_at": "2026-11-06T13:20:00+09:00" } ] }
```

`notify_count`는 알림을 받을 기기 수입니다. 기기 토큰 자체는 응답에 넣지 않습니다.

2-6. `POST /admin/api/pending/{id}/answer`

요청

필드	필수	설명
`question`	O	FAQ로 저장할 질문 (대표 문장을 다듬어 100자 이내)
`answer`	O	답변
`category`	O	카테고리
`campus`		캠퍼스
`ref_doc_id`		근거 보충 문서 (`document.doc_id`)

응답 201

```
{ "faq_id": 57, "indexed_at": "2026-10-12T10:02:11+09:00", "notified": 14 }
```

서버 처리 (승인 없이 바로, 1분 이내, PER-010)

1. `faq` 생성 (`pending_id`, `author_id` = 로그인한 사람) → `pending_question.status = 답변완료`, `handled_by`·`handled_at` 기록
2. 질문·답변을 임베딩해 벡터 DB `faq` 컬렉션에 추가 → `faq.indexed_at` 기록
3. 이 질문의 `notify_token` 전체에 FCM 데이터 메시지 발송 → 토큰 행 삭제

이미 답변한 질문이면 409입니다.

2-7. `POST /admin/api/pending/{id}/not-mine`

본문 없음 → 200 `{ "status": "담당아님" }`. 요약 메일과 대기 목록에서 빠지고, 관리자가 확인해 다른 경로로 안내합니다.

2-8. FAQ

메서드·경로	설명
`GET /admin/api/faqs?category=&q=`	목록 (`faq_id`, `question`, `category`, 작성자, `created_at`, `ask_count`)
`PUT /admin/api/faqs/{faq_id}`	수정 (`question`, `answer`, `category`, `campus`, `ref_doc_id`) → 즉시 재색인

메서드·경로	설명
`DELETE /admin/api/faqs/{faq_id}`	삭제 → 벡터 DB에서도 즉시 제거 (204)

2-9. 중간 관리자 🔴

메서드·경로	설명
`GET /admin/api/managers`	목록 (`email` , `name` , `affiliation` , `role` , `created_at` , `last_login_at` , `is_fixed`)
`POST /admin/api/managers`	등록 `{ "email", "name"?, "affiliation"? }` → 201. 이메일 형식 오류 400, 이미 있으면 409
`DELETE /admin/api/managers/{manager_id}`	삭제 → 204. 세션·인증 코드 함께 삭제(로그인 즉시 종료), 요약 메일 대상에서 빠짐. 고정 관리자는 403

2-10. 보충 문서 🔴

메서드·경로	설명
`POST /admin/api/documents`	`multipart/form-data` : `file` (PDF·HWP·TXT, 10MB 이하), `title` , `category` , `provider` → 202 `{ "doc_id": 88, "status": "재색인 대기" }`
`GET /admin/api/documents`	목록 (`title` , `provider` , `collected_at` , `status` (색인 완료/재색인 대기), 청크 수)
`DELETE /admin/api/documents/{doc_id}`	삭제 → 청크와 벡터 DB 임베딩도 삭제 (204)

2-11. 동의어 🔴

`GET /admin/api/synonyms` , `POST` `{ "term": "수정", "canonical": "수강정정" }` (이미 있으면 409), `DELETE /admin/api/synonyms/{synonym_id}`

2-12. 수집 현황 🔴

`GET /admin/api/collect/status`

```
{ "items": [ { "source_id": 3, "name": "학사공지", "interval_min": 180, "last_success_at": "2026-10-12T09:00:40+0
9:00",
               "last_status": "성공", "error_message": null } ] }
```

`POST /admin/api/collect/run` `{ "source_id": 3 }` (생략하면 전체) → 202. 결과는 `collect_log` 에 남습니다.

2-13. CSV 내려받기

`GET /admin/api/pending.csv` , `GET /admin/api/faqs.csv` → `text/csv; charset=utf-8` (엑셀에서 한글이 깨지지 않게 BOM 포함)

3. 서버 내부 작업 (스케줄러, HTTP 아님)

작업	시각	내용	테이블	요구사항
자주 바뀌는 소스 수집	3시간마다	학사·일반·학과 공지, 교외활동(온통청년 API)	`source`, `document`, `chunk`, `collect_log`	FR-014, PER-008
하루 1회 소스 수집	매일 03:00	학사일정, 통학버스(주말·방학 포함), 동아리, 캠퍼스맵, 학사요람	같음	FR-014, AVL-001
답변 대기 요약 메일	매일 09:00	대기 1건 이상일 때만 중간 관리자에게 발송. 서버가 꺼져 있었으면 켜진 뒤 그날 한 번	`pending_question`, `manager`, `digest_log`	FR-031, AVL-009
답변 대기 만료	매일 00:10	30일 지난 '대기'를 '만료'로, 알림 토큰 삭제	`pending_question`, `notify_token`	AVL-008
만료 데이터 정리	매일 00:20	만료된 인증 코드·세션 삭제	`auth_code`, `login_session`	FR-033

요약 메일 형식 (SCR-006-E)

- 제목: `[물어봐 캠퍼스] 답변 대기 6건`
- 본문: 같은 질문 수가 많은 순으로 질문 문장·카테고리·같은 질문 수 표, '답변 페이지에서 확인하기' 링크 (`/admin/pending`). '담당 아님' 질문은 빼고, 질문 문장은 이름·학번·전화번호 형태를 가린 문장을 씁니다.
- 발송: Python `smtplib` (SMTP), 발송 전용 계정. 계정 비밀번호는 환경 변수 (2.6)

4. 요구사항 추적

요구사항	API·작업
FR-001~005, 022, 023	1-2 질문하기
FR-006	1-4 추천 질문
FR-007	1-2 응답의 `sources[].source_url`
FR-014, 016	3 수집 작업, 2-10 보충 문서, 2-12 수집 현황
FR-019	이 문서 전체 (사용자 API `/api/v1`와 관리 API `/admin/api` 분리)
FR-020	1-3 마감 일정
FR-021	기기 로컬 알림 (API 없음, 1-2 `schedule`과 1-3 데이터 사용)
FR-024, 025	2-4, 2-5, 2-8, 2-13
FR-027	1-2 `clarify`
FR-028	1-2 처리 5단계
FR-029, 030	2-6, FCM 데이터 메시지, 1-5
FR-031	3 요약 메일
FR-032	2-9
FR-033	2-1, 2-2, 2-3
FR-008, 009	기기 저장 (API 없음)

5. 정할 것 (개발 전)

항목	상태
발송 전용 메일 계정 (Gmail 앱 비밀번호 또는 학교 메일 SMTP)	팀 결정 필요. 2-1·요약 메일이 이 계정을 씀
근거 유무를 가르는 유사도 기준값	테스트 질문셋 80개로 평가해 정함 (FR-002)
외부 접속 주소 (시연 기간 터널 서비스)	개발 단계에서 확인. 앱의 서버 주소와 요약 메일 링크에 들어감